

RPAL Interpreter

CS 3513 – Programming Languages

Basic Project Details

Language : C++ (C++17)
Executable : rpal20
Date : May 2026
Repository : https://github.com/praveen-de-silva/RPAL_Interpreter

Prepared by:

Bulagala D.W.K.G. : 230094U
De Silva B. K. P. : 230123K

1. Introduction

This report describes the design and implementation of a complete interpreter for the RPAL (Right-reference Pedagogic Algorithmic Language) programming language. The interpreter accepts an RPAL source file as input, evaluates it, and produces output that matches the reference `rpal.exe` implementation.

C++ is selected instead of Java because C++ provides greater control over memory management, pointers, tree structures, stacks, and runtime data representations. These features are useful when implementing compiler and interpreter components such as Abstract Syntax Trees, environments, closures, and the CSE machine. The implementation was done manually without using tools such as `lex` or `yacc`, and the final executable is built as `rpal20` according to the project requirements. It is built as a six-stage pipeline:

Lexer → *Screener* → *Parser* → *Standardizer* → *Flattener* → *CSE Machine*

Usage

```
./rpal20 <filename>      # evaluate program
./rpal20 -ast <filename>  # print Abstract Syntax Tree and exit
./rpal20 -st <filename>   # print Standardized Tree and exit
```

Build

```
make          # builds the rpal20 executable
make clean    # removes the executable
```

2. Program Structure

The interpreter follows a classic linear compiler pipeline. Each stage transforms its input into a data structure consumed by the next stage.

Stage	Module	Input	Output	Author
1	Lexer	Source file (text)	vector<Token>	230123K
2	Screener	vector<Token> (raw)	vector<Token> (filtered)	230123K
3	Parser	vector<Token>	ASTNode* (AST)	230094U
4	Standardizer	ASTNode* (AST)	ASTNode* (ST)	230123K
5	Flattener	ASTNode* (ST)	vector<vector<CSENode>>>	230094U
6	CSE Machine	vector<vector<CSENode>>>	Program output (stdout)	230094U, 230123K

Directory Layout

```

./
├── main.cpp
├── Makefile
├── lexer/
│   ├── Token.h / Token.cpp
│   ├── Lexer.h / Lexer.cpp
│   └── Screener.h / Screener.cpp
├── parser/
│   ├── ASTNode.h / ASTNode.cpp
│   └── Parser.h / Parser.cpp
├── standardizer/
│   └── Standardizer.h / Standardizer.cpp
├── flattener/
│   └── Flattener.h / Flattener.cpp
└── cse_machine/
    ├── StackValue.h / StackValue.cpp
    ├── Environment.h / Environment.cpp
    └── CSEMachine.h / CSEMachine.cpp

```

The directory layout shown above represents the original development structure used during implementation. For the final Moodle submission, the files were rearranged into a flat directory structure as required by the submission guidelines. All source files, header files, the Makefile, and the report were placed directly in the submitted zip file without nested folders. The original development structure was not changed; only a separate submission copy was prepared.

3. Module Descriptions and Function Prototypes

3.1 Token (lexer/Token.h, Token.cpp)

Defines the TokenType enum and the Token data structure used throughout the pipeline.

```
enum class TokenType {
    IDENTIFIER, INTEGER, STRING, OPERATOR,
    PUNCTUATION, KEYWORD, SPACES, COMMENT, END_OF_FILE
};

struct Token {
    TokenType type;
    std::string value;
    Token(TokenType t, std::string v);
    std::string typeName() const;
};
```

3.2 Lexer (lexer/Lexer.h, Lexer.cpp)

Stage 1. Reads the entire source file and converts it into a raw token stream (including SPACES and COMMENT tokens later removed by the Screener). Recognises 21 RPAL keywords.

```
class Lexer {
public:
    explicit Lexer(const std::string& filename);
    std::vector<Token> tokenize();
private:
    char currentChar() const;
    char peek() const;
    void advance();
    bool isLetter(char c) const;
    bool isDigit(char c) const;
    bool isOperatorSymbol(char c) const;
    Token readIdentifierOrKeyword();
    Token readInteger();
    Token readString();
    Token readOperator();
    Token readSpaces();
    Token readComment();
};
```

3.3 Screener (lexer/Screener.h, Screener.cpp)

Stage 2. Filters out SPACES and COMMENT tokens, leaving only meaningful tokens for the parser.

```
class Screener {
public:
    std::vector<Token> filter(const std::vector<Token>& tokens);
};
```

3.4 ASTNode (parser/ASTNode.h, ASTNode.cpp)

Node type used for both the Abstract Syntax Tree (AST) and the Standardized Tree (ST). Children are stored as a linked list via child/sibling pointers.

```
class ASTNode {
public:
    std::string type;        // e.g. "let", "gamma", "IDENTIFIER"
    std::string value;       // leaf value e.g. "42", "x", "'hello'"
    ASTNode* child;
    ASTNode* sibling;
    ASTNode(std::string t, std::string v = "");
    ~ASTNode();
    void print(int depth = 0) const;
};
```

3.5 Parser (parser/Parser.h, Parser.cpp)

Stage 3. Hand-written recursive descent parser implementing the full RPAL grammar. Uses a tree-building stack; each grammar rule calls buildTree(type, n) to pop n nodes and attach them as children of a new parent.

```
class Parser {
public:
    Parser(const std::vector<Token>& tokenStream);
    ASTNode* parse();
private:
    const Token& nextToken() const;
    void read(const std::string& expectedValue = "");
    void buildTree(const std::string& type, int numChildren);
    // 21 recursive descent functions:
    void parseE(); void parseEw(); void parseT(); void parseTa();
    void parseTc(); void parseB(); void parseBt(); void parseBs();
    void parseBp(); void parseA(); void parseAt(); void parseAf();
    void parseAp(); void parseR(); void parseRn(); void parseD();
    void parseDa(); void parseDr(); void parseDb(); void parseVb();
    void parseVl();
};
```

3.6 Standardizer (standardizer/Standardizer.h, Standardizer.cpp)

Stage 4. Transforms the AST into a Standardized Tree (ST) by applying nine bottom-up rewrite rules in-place.

```
class Standardizer {
public:
    ASTNode* standardize(ASTNode* node); // recursive entry point
private:
    ASTNode* makeNode(const std::string& type, const std::string& value =
    "");
    ASTNode* copyTree(ASTNode* node); // deep copy (needed for rec)
    ASTNode* getChild(ASTNode* node, int n); // 0-indexed child access
    int countChildren(ASTNode* node);
    void standardizeLet(ASTNode* node); // Rule 1
    void standardizeWhere(ASTNode* node); // Rule 2
    void standardizeFcnForm(ASTNode* node); // Rule 3
    void standardizeLambda(ASTNode* node); // Rule 4
    void standardizeAnd(ASTNode* node); // Rule 5
    void standardizeRec(ASTNode* node); // Rule 6
    void standardizeWithin(ASTNode* node); // Rule 7
    void standardizeAt(ASTNode* node); // Rule 8
    void standardizeEmptyParam(ASTNode* node); // Rule 9
};
```

The nine transformation rules:

- **Rule 1 (let):** let x=E in P → gamma (lambda x P) E
- **Rule 2 (where):** P where x=E → gamma (lambda x P) E
- **Rule 3 (fcn_form):** f x1 x2 = E → = f (lambda x1 (lambda x2 E))
- **Rule 4 (lambda):** lambda x1 x2 E → lambda x1 (lambda x2 E)
- **Rule 5 (and):** and (=x1 E1) (=x2 E2) → = (tau x1 x2) (tau E1 E2)
- **Rule 6 (rec):** rec (= f E) → = f (Y* (lambda f E))
- **Rule 7 (within):** within (=x1 E1) (=x2 E2) → = x2 (gamma (lambda x1 E2) E1)
- **Rule 8 (@):** E @n R → gamma (gamma n E) R
- **Rule 9 (empty):** f () = E → = f (lambda dummy E)

3.7 Flattener (flattener/Flattener.h, Flattener.cpp)

Stage 5. Converts the ST into delta arrays consumed by the CSE Machine. Each lambda body and conditional branch becomes its own numbered delta array.

```
class Flattener {
public:
    Flattener();
    void flatten(ASTNode* root);
    const std::vector<std::vector<CSENode>>& getDeltas() const;
    void print() const;
private:
    void flattenNode(ASTNode* node, int currentDelta);
    std::vector<std::vector<CSENode>> deltas;
    int nextDelta;
};
```

3.8 StackValue (cse_machine/StackValue.h, StackValue.cpp)

Represents any value that can appear on the CSE Machine's value stack.

```
enum class ValueType {
    INTEGER, STRING, BOOL, NIL, DUMMY,
    TUPLE, CLOSURE, ETA, BUILTIN, PARTIAL
};

class StackValue {
public:
    static StackValue makeInt      (int v);
    static StackValue makeStr      (const std::string& v);
    static StackValue makeBool     (bool v);
    static StackValue makeNil      ();
    static StackValue makeDummy    ();
    static StackValue makeTuple    (std::vector<StackValue> elems);
    static StackValue makeClosure(int deltaIdx, std::vector<std::string>
vars, int envIdx);
    static StackValue makeEta      (int deltaIdx, std::vector<std::string>
vars, int envIdx);
    static StackValue makeBuiltin(const std::string& name);
    static StackValue makePartial(const std::string& name, StackValue
firstArg);
    std::string toString() const;
};
```

3.9 Environment (cse_machine/Environment.h, Environment.cpp)

Represents a single lexical scope. Environments form a parent chain enabling lexical scoping.

```
class Environment {
public:
    Environment(int idx, int parentIdx);
    void bind      (const std::string& name, const StackValue& val);
    bool lookupLocal(const std::string& name, StackValue& out) const;
    int  getParentIndex() const;
};
```

3.10 CSE Machine (cse_machine/CSEMachine.h, CSEMachine.cpp)

Stage 6. Evaluates the delta arrays using the Control–Stack–Environment model. Implements all 13 CSE rules and 12 built-in functions.

```
class CSEMachine {
public:
    explicit CSEMachine(const std::vector<std::vector<CSENode>>&
deltaStructures);
    void evaluate();
private:
    // Helpers
    int      newEnv  (int parentIdx);
    StackValue lookup (const std::string& name, int envIdx) const;
    void      pushDelta(int idx);
    // Rules 1,2,3,4,5,8,11,12 (230094U)
```

```

void rule1_name      (const CSENode& node);
void rule2_lambda    (const CSENode& node);
void rule3_4_gamma   ();
void rule5_envMarker (const CSENode& node);
void rule8_beta      ();
void rule11_ystar     (StackValue& lambda);
void rule12_eta       (StackValue& eta, StackValue& rand);
// Rules 6,7,9,10,13  (230123K)
void rule6_binaryOp   (const CSENode& node);
void rule7_unaryOp    (const CSENode& node);
void rule9_tau        (const CSENode& node);
void rule10_tupleIndex(StackValue& tuple, StackValue& idx);
void rule13_builtin   (StackValue& rator, StackValue& rand);
// Built-ins  (230123K)
void      builtinPrint      (const StackValue& arg);
StackValue builtinOrder     (const StackValue& arg);
StackValue builtinStem      (const StackValue& arg);
StackValue builtinStern     (const StackValue& arg);
StackValue builtinConc      (const StackValue& s1, const StackValue&
s2);
StackValue builtinIsinteger  (const StackValue& arg);
StackValue builtinIsstring   (const StackValue& arg);
StackValue builtinIstruthvalue(const StackValue& arg);
StackValue builtinIstuple    (const StackValue& arg);
StackValue builtinIsfunction (const StackValue& arg);
StackValue builtinArity      (const StackValue& arg);
StackValue builtinNull       (const StackValue& arg);
};

```

4. CSE Machine – Evaluation Rules

The CSE Machine operates on three data structures:

1. Control (C): instruction stack
2. Stack (S): value stack
3. Environment (E): current active environment index.

Rule	Trigger	Action	Author
1	IDENTIFIER on control	Lookup in env chain; push value to stack	230094U
2	LAMBDA on control	Create closure (delta, vars, env); push to stack	230094U
3	GAMMA + single-param closure	New env, bind one param, push delta body	230094U
4	GAMMA + multi-param closure	New env, bind params from tuple	230094U
5	ENV marker on control	Restore environment; preserve return value	230094U
6	Binary OPERATOR	+ - * / ** gr ge ls le eq ne or & aug	230123K
7	Unary OPERATOR	neg / not — pop one value, push result	230123K
8	BETA on control	Pop boolean; push correct branch delta	230094U
9	TAU(n) on control	Pop n values; create tuple; push to stack	230123K
10	GAMMA + tuple on stack	Pop integer index; push tuple element (1-based)	230123K
11	GAMMA + Y* on stack	Wrap lambda in eta (recursive) closure	230094U
12	GAMMA + eta closure	Unwrap one recursion step; create two new envs	230094U
13	GAMMA + built-in on stack	Dispatch to built-in function implementation	230123K

Built-in Functions

Function	Description
Print / print	Print value to stdout (strings without surrounding quotes)
Order	Number of elements in a tuple (0 for nil)
Stem	First character of a string
Stern	String without its first character
Conc	Concatenate two strings (curried)
Isinteger	Type predicate: true if integer
Isstring	Type predicate: true if string
Istruthvalue	Type predicate: true if boolean
Istuple	Type predicate: true if tuple or nil
Isfunction	Type predicate: true if closure or built-in
Arity	Number of parameters of a closure
null	True if nil or empty tuple

5. Sample Test Cases and Output

5.1 Comprehensive (factorial, boolean, arithmetic, string)

Input:

```
let
  logic_test = (true & false) or (not false)
  and
  arith_test = 2 ** 3 + 16 / 2 - 1
  and
  string_test = 'Hello RPAL'
  and
  rec factorial n = n eq 0 -> 1 | n * factorial (n - 1)
in
  Print (factorial 5, logic_test, arith_test, string_test)
```

Output:

```
(120, true, 15, 'Hello RPAL')
```

5.2 Infix @ operator

Input:

```
let add x y = x + y
in Print (3 @add 4)
```

Output:

```
7
```

5.3 Recursive sum

Input:

```
let Sum(A) = Psum (A, Order A)
where rec Psum (T, N) = N eq 0 -> 0
      | Psum(T, N-1) + T N
in Print (Sum (1,2,3,4,5))
```

Output:

```
15
```

5.4 String operations (Stem, Stern, Conc)

Input:

```
let s = 'hello'
in Print (Stem s, Stern s, Conc 'foo' 'bar')
```

Output:

```
('h', 'ello', 'foobar')
```

5.5 Palindrome checker (Stem + Stern + Conc + rec)

Input:

```
let rec Reverse S = S eq '' -> ''
                  | Conc (Reverse (Stern S)) (Stem S)
in
let IsPalindrome S = (S eq (Reverse S)) -> 'true' | 'false'
in
Print (IsPalindrome 'madam', IsPalindrome 'hello')
```

Output:

```
('true', 'false')
```

5.6 Tuple operations (aug, nil, Order, null)

Input:

```
let t = nil aug 10 aug 20 aug 30
in Print (t, Order t, null nil, null t)
```

Output:

```
((10, 20, 30), 3, true, false)
```

5.7 Simultaneous definitions (and) + within

Input:

```
let a = 1 and b = 2 and c = 3 in Print (a+b+c)
```

Output:

```
6
```

5.8 Type predicates

Input:

```
Print (Isinteger 42, Isstring 'hi', Istruthvalue false, Istuple (1,2,3))
```

Output:

```
(true, true, true, true)
```

6. Division of Responsibilities

Component	230094U	230123K
Lexer / Screener		✓
Parser / ASTNode	✓	
Standardizer (all 9 rules)		✓
Flattener	✓	
CSE Rules 1, 2, 3, 4, 5	✓	
CSE Rules 8, 11, 12 (beta/Y*)	✓	
CSE Rules 6, 7 (operators)		✓
CSE Rules 9, 10 (tau/tuple)		✓
CSE Rule 13 + all built-ins		✓
Environment / StackValue	✓ (structure)	✓ (toString)

7. Conclusion

This project successfully implements a complete interpreter for the RPAL programming language. The interpreter reads an RPAL source program, converts it into tokens, builds the Abstract Syntax Tree, standardizes the tree, flattens it into control structures, and evaluates it using the CSE machine.

Through this project, we gained practical experience in the internal working of programming languages, including lexical analysis, recursive descent parsing, tree transformation, closures, environments, recursion, tuples, built-in functions, and abstract machine execution.

The final program can be compiled using the provided Makefile and executed using the required command format. The implementation produces output that matches the expected RPAL program outputs.